



UNIVERSIDAD DE LOS LAGOS

Laboratorio N°1: midiendo las falacias

Semana 2 · Taller online · Viernes 28-08-2026 · 14:15–16:00

FDICI25 · INFO35

Ingeniería Civil en Informática · 2026

Campus Osorno

Av. Fuchslocher 1305
Teléfono +56 64 2333 000
Fax +56 64 2333 774
Osorno, Chile

Campus Puerto Montt

Camino a Chingihue Km 6
Teléfono +56 65 2322 536
Puerto Montt, Chile

Sede Santiago

República 517
Barrio Universitario
Teléfono +56 02 2675 3057
Santiago, Chile

Sede Chiloé

Ubaldo Mansilla Barrientos 131
Teléfono 56 65 2322 409
Castro, Chile
Eleuterio Ramírez 348
Teléfono +56 65 2322 476
Ancud, Chile



www.ulagos.cl

Objetivo, entregables y cronograma del laboratorio

Objetivo

- Observar y medir cómo la latencia y la pérdida de paquetes degradan un cliente-servidor real que corre en dos nodos.
- Registrar con rigor: cada número anotado debe poder reproducirse.

Entregables (por equipo, al repositorio)

- semana02/mediciones.md completo, con commit antes de las 16:00.
- Informe breve (máx. 2 planas) como trabajo autónomo: plazo miércoles 02-09, 23:59.

Hora	Paso	Qué se hace
14:15–14:30	0 · Preparación	copiar semana02/ al repo del equipo, docker compose up, verificar que cliente y servidor se ven
14:30–14:45	1 y 2 · Línea base	ping, iperf3 y 100 llamadas con la red sana · CP1 al chat: RTT y throughput
14:45–15:05	3 · Latencia	tc netem delay 50 / 200 / 500 ms y repetir las 100 llamadas · CP2 al chat: total con 200 ms
15:05–15:15	Pausa	
15:15–15:35	4 · Pérdida y falla	loss 1 / 5 / 20%; falla provocada con loss 100% · CP3 al chat: qué hizo el cliente
15:35–15:50	5 · Registro	mediciones.md, commit y push · CP4 al chat: enlace al commit
15:50–16:00	Cierre	Limpieza, asignación del informe. La sesión termina a las 16:00 en punto

Cómo trabajamos hoy: dinámica de la sesión online

Conexión y equipo

- Ambos integrantes conectados a la videollamada del curso durante toda la sesión.
- El equipo se coordina por su propio canal (llamada o chat aparte): uno ejecuta en su notebook, el otro registra.
- Micrófonos apagados; las dudas se piden por el chat o levantando la mano.

Cómo pedir ayuda

- Escribir en el chat: equipo + paso + el error textual (copiado, no descrito).
- El docente llama al equipo y le pide compartir pantalla para revisar en vivo.
- Mientras esperan ayuda, el otro integrante adelanta el registro en mediciones.md.

Checkpoints obligatorios (van al chat de la videollamada, un mensaje por equipo)

- **CP1 (14:35):** "Equipo N — RTT base X ms, iperf3 Y Gbit/s". **CP2 (14:50):** "Equipo N — total con 200 ms: X s".
- **CP3 (15:30):** "Equipo N — con loss 100% el cliente ...". **CP4 (15:50):** "Equipo N — commit: enlace".
- Los checkpoints reemplazan la vuelta por los puestos del docente: son la evidencia de avance y la asistencia de la sesión.

Qué vamos a medir: dos nodos en la red sd_net



Convenciones

- red: `sd_net`
- nodos: servidor, cliente
- puerto: 5000
- imagen: `python:3.12-slim`

Herramienta	Qué mide o hace	Falacia asociada
ping	Tiempo de ida y vuelta (RTT) a nivel ICMP	2 · La latencia es cero
iperf3	Ancho de banda efectivo TCP entre los dos nodos	3 · El ancho de banda es infinito
tc netem	Inyecta retardo y pérdida de paquetes en la interfaz eth0 del cliente	1 y 2 · La red es confiable / la latencia es cero
cliente.py	100 llamadas remotas secuenciales, tiempo por llamada y total	Lo que siente la aplicación

Paso 0 · Preparación (14:15–14:30)

PowerShell (Windows) o bash (Linux/macOS), con Docker iniciado

```
$ cd ruta/del/repositorio-equipo      # Windows: ruta\del\...
# descomprimir semana02.zip (plataforma) dentro del repo
$ cd semana02
$ docker compose up -d --build # 1ª vez: 1-2 min
$ docker compose ps           # cliente Up, servidor Up

# atajo para el resto del lab: "c" = ejecutar dentro del cliente
PS> function c { docker compose exec cliente @args } # Windows
$ c() { docker compose exec cliente "$@"; }         # Linux/macOS
```

Verificación: el cliente ve al servidor por nombre

```
$ c ping -c 2 servidor
64 bytes from servidor (172.x.x.x): icmp_seq=1 time=0.1 ms
```

Contenido de semana02.zip

- docker-compose.yml (no modificar)
- Dockerfile
- servidor.py (caja negra esta semana)
- cliente.py (léanlo: 40 líneas)
- mediciones.md (lo completan ustedes)

Si ping falla: docker compose down; borrar una sd_net vieja de la semana 1 con docker network rm sd_net; volver a levantar.

Paso 1 · Línea base: ping e iperf3 (14:30–14:37)

RTT con ping

```
$ c ping -c 10 servidor
...
10 packets transmitted, 10 received, 0% loss
rtt min/avg/max/mdev = 0.05/0.08/0.15/0.02 ms
```

iperf3 (servidor en 2º plano con -d)

```
$ docker compose exec -d servidor iperf3 -s
$ c iperf3 -c servidor -t 5
...
[ ID] Interval      Transfer   Bitrate
[  5] 0.00-5.00 sec 20 GBytes 35 Gbits/sec
```

Anotar en mediciones.md → Paso 1

- RTT promedio de ping (ms) y throughput de iperf3 (Gbit/s). Son números de un bridge virtual: esto es "la red perfecta" que las falacias suponen.
- Pregunta para el registro: ¿qué tan lejos están estos valores de los de la tabla de latencias del jueves?

Atención con Docker Desktop (Windows/macOS): el primer iperf3 puede dar menos por el arranque de la VM; repetir una vez y quedarse con la segunda medición. En Linux nativo no aplica.

Paso 2 · 100 llamadas remotas con la red sana (14:37–14:45)

Ejecutar el cliente (y una segunda vez con 1000 llamadas)

```
$ c python cliente.py
llamadas=100 total=0.045s promedio=0.4ms min=0.2ms max=1.8ms

$ c env N_LLAMADAS=1000 python cliente.py
llamadas=1000 total=0.41s promedio=0.4ms min=0.2ms max=2.1ms
```

Ver el otro lado: qué registró el servidor

```
$ docker compose logs servidor
[servidor] escuchando en 0.0.0.0:5000
[servidor] conexión desde ('172.x.x.3', 4xxxx)
[servidor] ('172.x.x.3', 4xxxx) cerró la conexión
```

Qué observar

- El total es $N \times \text{promedio}$: cada llamada espera a la anterior.
- $\text{max} > \text{promedio}$: la red ya no es constante ni siquiera en local.

Anotar en mediciones.md → fila "0 ms (base)":
total, promedio y máximo. Un integrante ejecuta, el otro anota. Rotar en el paso 3.

Lo que hace cliente.py (léanlo antes de seguir)

cliente.py (fragmento)

```
HOST = os.getenv("SERVIDOR_HOST", "servidor")
N = int(os.getenv("N_LLAMADAS", "100"))
TIMEOUT = float(os.getenv("TIMEOUT_S", "0")) or None

with socket.create_connection((HOST, PUERTO),
                             timeout=TIMEOUT) as s:
    f = s.makefile("rwb")
    for i in range(1, N + 1):
        t0 = time.perf_counter()
        f.write(f"PING {i}\n".encode()); f.flush()
        respuesta = f.readline() # <- aquí ESPERA a la red
        tiempos_ms.append((time.perf_counter() - t0) * 1000)
```

Tres decisiones de diseño

- Una sola conexión para las 100 llamadas (sin handshake por llamada).
- Llamadas secuenciales: no envía la siguiente hasta recibir la anterior.
- TIMEOUT=0 por defecto: el programa confía en que la red siempre responde. Hoy verán por qué eso es un error.

En la semana 4 escribirán su propio cliente y servidor; este es el punto de partida que van a superar.

Paso 3 · Inyectar latencia con tc netem (14:45–15:05)

50 ms

```
$ c tc qdisc add dev eth0 root netem delay 50ms
$ c ping -c 3 servidor          # debe mostrar ~50 ms
$ c python cliente.py
llamadas=100 total=5.06s promedio=50.5ms min=50.2ms max=52.0ms
```

200 ms y 500 ms: "change", no "add"

```
$ c tc qdisc change dev eth0 root netem delay 200ms
$ c python cliente.py
$ c tc qdisc change dev eth0 root netem delay 500ms
$ c python cliente.py
```

Antes de ejecutar

- Calculen cuánto debería tardar el total. Luego comparen.
- Anoten total, promedio y máximo por nivel.

Pregunta guía (va al informe): ¿la degradación es lineal con la latencia? ¿Qué pasa con el máximo? ¿Cambió algo en el servidor? (revisen docker compose logs servidor).

Paso 4 · Pérdida de paquetes (15:15–15:28)

Sin retardo, solo pérdida (netem reemplaza la regla anterior)

```
$ c tc qdisc change dev eth0 root netem loss 1%
$ c iperf3 -c servidor -t 5
$ c python cliente.py
# repetir con loss 5% y loss 20%
```

Lo que suele verse

- iperf3 cae de decenas de Gbit/s a unos pocos con 5% de pérdida.
- cliente.py sigue "funcionando", pero el máximo se dispara: TCP retransmite en silencio.

Pérdida	Throughput iperf3	Total cliente (100)	Qué explica
1%	baja notoriamente	casi igual, con picos	TCP recupera con reenvíos rápidos
5%	cae a una fracción	más lento, máximo alto	Control de congestión reduce la ventana
20%	casi inutilizable	varios segundos	Retransmisiones con esperas de 200 ms+

Valores orientativos; anoten los reales en mediciones.md → Paso 4. La transparencia de fallos de TCP "funciona", pero a costa del tiempo.

Provocación de falla · loss 100% y un cliente sin timeout

1. La red desaparece, el programa no se entera (15:28)

```
$ c tc qdisc change dev eth0 root netem loss 100%
$ c python cliente.py
    <- nada. Esperar 60 s mirando la pantalla.
^C (Ctrl+C para abortar)
```

2. El mismo cliente, con una sola variable distinta

```
$ c env TIMEOUT_S=3 python cliente.py
[cliente] TIMEOUT tras 3.0s esperando al servidor
(0 llamadas completadas)
```

Registrar (fila 100%)

- ¿Cuánto esperaron antes de abortar? Sin timeout, TCP seguiría reintentando hasta ~15 min.
- ¿El servidor se enteró de algo? (docker compose logs servidor).
- ¿Qué debería hacer la aplicación después del timeout: reintentar, avisar, abortar?

Este es el momento del laboratorio. Un programa que espera para siempre asumió la falacia 1. Todo cliente que escriban desde la semana 4 en adelante debe tener un timeout y una decisión explícita sobre qué hacer cuando vence.

Plan B · Si tc responde "Operation not supported"

Por qué puede pasar

- Algunos kernels de WSL2 (Windows) o de la VM de Docker Desktop (macOS) no traen sch_netem; en Linux nativo casi nunca falta.
- Docker Desktop sin cap_add NET_ADMIN (revisar que no editaron el compose).

Qué cambia y qué no

- La latencia la simula el servidor (sleep por llamada) en vez de la red.
- Se pierden las mediciones con ping/iperf3; las de cliente.py se mantienen.
- Anotar en mediciones.md: "Plan B, latencia simulada en el servidor".

Activar el Plan B (la variable ya existe en docker-compose.yml)

```
$ c tc qdisc add dev eth0 root netem delay 50ms
RTNETLINK answers: Operation not supported      # <- Plan B
PS> $env:SIMULAR_LATENCIA_MS="200"              # Windows
$ export SIMULAR_LATENCIA_MS=200                 # Linux/macOS
$ docker compose up -d --force-recreate servidor
$ c python cliente.py
llamadas=100 total=20.1s promedio=200.6ms ...
```

Paso 5 · Registro, commit y push (15:35–15:50)

semana02/mediciones.md (plantilla incluida en el repositorio)

```
# Mediciones – Laboratorio N°1 (Semana 2)
## Pasos 2 y 3 – Latencia inyectada (100 llamadas)
| Latencia | Total (s) | Prom (ms) | Máx (ms) | ¿Esperado? |
| 0 ms     | 0.045     | 0.4       | 1.8       | sí         |
| 200 ms   | 20.1      | 200.6     | 203.1     | sí, N×RTT  |
...
## Falacia que asumimos sin advertirlo
El cliente no tiene timeout: asume que la red es
confiable (falacia 1).
```

Criterios de un buen registro

- Unidades en cada número.
- Condición exacta de cada fila (delay, loss, N).
- Una observación por fila, no solo el número.
- Lo que salió distinto a lo esperado, dicho explícitamente.

Antes de las 15:50

```
$ git add semana02/mediciones.md
$ git commit -m "Semana 2: mediciones de latencia y perdida (Lab 1)"
$ git push
```

Limpieza del entorno (15:50–15:55)

Dejar el laboratorio como lo encontraron

```
$ c tc qdisc del dev eth0 root # quitar netem
$ docker compose down -v      # contenedores, red sd_net y volúmenes
$ docker network ls           # sd_net no debe aparecer
$ docker compose ps           # vacío
```

Por qué importa

- Una regla netem olvidada deja lento TODO el tráfico de ese contenedor; si retoman el material otro día, las mediciones salen malas sin explicación.
- La semana 4 levantará sd_net de nuevo: una red huérfana provoca "network already exists".

Antes de irse

- Confirmar en GitHub que el push llegó (mediciones.md visible).
- Registrar en el equipo quién redacta el informe y quién lo revisa.

Trabajo autónomo · Informe breve (máx. 2 planas, por equipo)

Sección	Qué debe contener	Extensión
1. Entorno y método	Equipo, sistema operativo y Docker, cómo se midió (pasos 1–4), si se usó Plan B	¼ plana
2. Resultados	Tabla de mediciones.md y un gráfico total vs latencia (puede ser el de la semana o hecho a mano)	½ plana
3. Observado vs esperado	Dónde coincidieron los cálculos y dónde no; explicación de las diferencias	½ plana
4. La falacia que asumimos	Cuál, en qué línea de cliente.py se ve, y qué cambio la corregiría	½ plana
5. Relación con la investigación	Qué falacia ha afectado a la aplicación que investigan para la Evaluación 1	¼ plana

Entrega: semana02/informe.md (o PDF) en el repositorio del equipo, miércoles 02-09 a las 23:59. **Carácter:** formativo, sin nota; la retroalimentación se entrega el jueves 03-09 y el análisis se reutiliza en la prueba parcial del 25-09.



UNIVERSIDAD DE LOS LAGOS

Trabajo autónomo y próxima clase

Próxima clase: jueves 03-09 (online) — Modelos de sistemas distribuidos, desafíos y teorema CAP.



Para la próxima clase:

1. Informe breve del laboratorio: miércoles 02-09, 23:59.
2. Avance de la investigación (Ev. 1, jueves 10-09).
3. Lectura: Tanenbaum & Van Steen (2017), cap. 1 completo.
4. Traer una duda sobre el laboratorio para abrir el jueves.

Campus Osorno

Av. Fuchslocher 1305
Teléfono +56 64 2333 000
Fax +56 64 2333 774
Osorno, Chile

Campus Puerto Montt

Camino a Chingihue Km 6
Teléfono +56 65 2322 536
Puerto Montt, Chile

Sede Santiago

República 517
Barrio Universitario
Teléfono +56 02 2675 3057
Santiago, Chile

Sede Chiloé

Ubaldo Mansilla Barrientos 131
Teléfono 56 65 2322 409
Castro, Chile
Eleuterio Ramírez 348
Teléfono +56 65 2322 476
Ancud, Chile



www.ulagos.cl